

软件过程重构大作业二

《软件过程改进方案设计文档》

--软件学院-2023141461121-孔垂骄

项目基础： 四川大学教务系统教务数据异步解析与 API 网关项目

一、 过程改进概述与核心目标

在传统的教务数据异步客户端开发中，团队面临着数据结构（如 xkx 课表）解析繁琐、Redis 限流与跨域（CORS）中间件边界测试困难等工程痛点。本改进方案旨在突破传统瀑布或纯敏捷模型中“人工驱动”的效率瓶颈，将生成式 AI 技术与现有的软件生命周期（SDLC）深度融合，构建一个“AI 辅助分析 - 智能编码 - 自动化验证”的新型闭环软件过程。

核心目标：

- 缩短交付周期：** 将复杂业务数据模型（Pydantic）的构建时间缩短 80%。
- 提升质量基线：** 引入大模型辅助的测试驱动开发（TDD），使网关异常处理与缓存策略的边界测试覆盖率达到 90% 以上 [2]。
- 沉淀智能资产：** 建立结构化的 Prompt 模板库，形成团队级别的 AI 协同规范。

二、 组织角色与职责重构

为了适配 AI 技术的引入，必须对原有软件过程中的角色职责进行调整，新增并兼任以下关键节点角色：

- AI 工具实施与架构师 (AI Tool Specialist & Architect)**
 - 职责：** 负责大语言模型与本地 IDE 的集成与配置；制定《项目数据脱敏与 AI 提问规范》；设计教务数据解析的标准化 Prompt 模板。
 - 人员映射：** 由原项目的核心全栈开发兼任。
- AI 代码与安全审查员 (AI Code & Security Reviewer)**
 - 职责：** 负责对 AI 生成的业务逻辑进行人工复核（Human-in-the-loop）。研究表明，AI 生成的代码可能包含隐蔽的安全漏洞（如弱凭证或注入风险）[3]，因此该角色必须严格审查是否存在 API 密钥硬编码及数据越权访问风险。
 - 人员映射：** 由原项目的架构师或交叉评审的开发人员兼任。

3. 智能测试分析师 (AI Test Analyst)

- **职责：** 利用 AI 测试工具解析 OpenAPI/Swagger 文档，生成针对 Redis 限流、7 天 TTL 缓存击穿等边缘场景的自动化测试脚本。

三、 核心过程改进设计与流程图

针对第一阶段报告识别出的关键痛点，我们对原有的软件过程活动进行了重构，并在关键流转节点强制植入了 AI 辅助环节。

1. 软件实现过程（编码）的改进：AI 驱动的复杂数据解析

- **原有流程：** 获取教务 HTML/JSON -> 人工分析结构 -> 手写正则/DOM 解析 -> 手写实体类映射。
- **改进后流程：** 获取教务源数据 -> [AI 节点] 调用脱敏 Prompt 模板 -> [AI 节点] 自动生成 httpx 异步解析逻辑与 Pydantic 校验模型 -> 人工 Code Review -> 集成。
- **质量标准：** AI 生成的模型必须包含完整的字段验证，且必须通过静态类型检查。

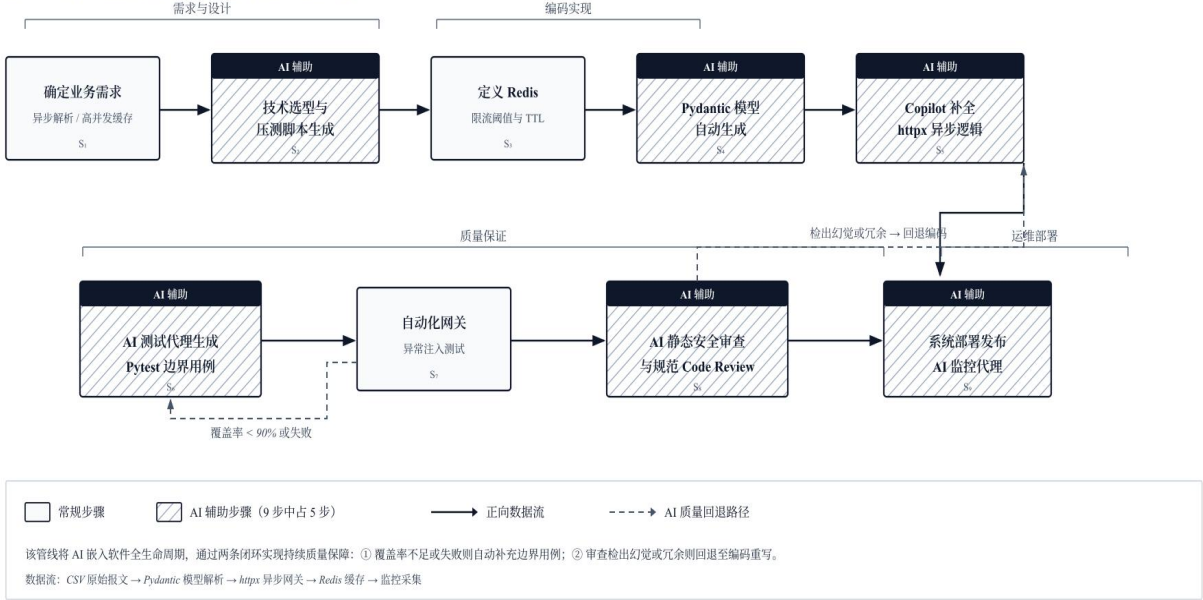
2. 软件测试过程的改进：基于 AI 的网关与异常测试

- **原有流程：** 开发完成跨域/分层异常模块 -> 人工构思异常场景 -> 手动编写 pytest 脚本 -> 覆盖率不达标反复修改。
- **改进后流程：** 模块开发完成 -> [AI 节点] 输入异常基类与接口文档 -> [AI 节点] 自动生成全边界等价类 pytest 测试集合 -> 运行测试 -> 人工补充特定教务逻辑。
- **质量标准：** 根据中国信通院的智能测试标准 [4]，AI 生成的测试用例必须覆盖核心的异常态。

3. 改进后的 AI 融合软件过程流转图 (Mermaid)

以下是本项目重构后的敏捷迭代生命周期图，清晰标明了 AI 技术的切入点与输出物验证机制：

图 1 AI 辅助的软件研发管线及闭环质量门禁



改进环节	推荐 AI 工具	核心功能与集成方式	预期效率提升指标
数据建模解析	GPT-4o	通过标准模板向 API 发送脱敏教务数据源，回传 Pydantic 模型。	解析与实体类编写耗时降低 80%。
网关编码补全	GitHub Copilot	替代传统 IDE 内置提示，开启内联自动补全与快捷键代码重构。	网络请求代码开发效率提升 50%。
异常用例生成	CodiumAI	作为 IDE 插件，选中 CORS 中间件代码，一键生成 Pytest 单元测试集 [2]。	测试代码编写时间缩短 60%。

五、 风险评估与应对策略

1. 模型幻觉与代码冗余风险 (Code Churn)

- **风险描述：** GitClear 针对 1.5 亿行代码的最新研究表明，过度依赖 AI 编程助手会导致代码“流动率 (Code Churn)”上升，即生成大量虽然能跑通但不符合 DRY (Don't Repeat Yourself) 原则的重复冗余代码 [1]。
- **应对策略 (Human-in-the-loop)：** 确立“AI 建议，人类决策”的核心原则。所有由 AI 生成的核心中间件与解析代码，必须强制通过静态扫描并经过 AI 代码审查员的逻辑复盘。

2. 数据的安全与安全漏洞引入风险

- **风险描述：** 学术界对 GitHub Copilot 的安全性评估指出，在特定场景下，AI 可能会生成包含 CWE 漏洞（如 SQL 注入、跨站脚本或硬编码凭证）的代码 [3]。同时，将教务系统真实数据喂给云端模型存在严重的隐私泄露风险。
- **应对策略：** 强制执行《本地数据脱敏规范》。在使用 AI 分析 xkx 课表结构前，必须使用本地脚本将真实学号、Cookie 替换为占位符。敏感 API 密钥必须通过环境变量注入，严禁进入 Prompt 历史记录。

六、 总结与展望

本设计方案以四川大学教务数据抓取及网关项目为载体，系统性地重构了传统的软件生命周期。通过在编码实现阶段引入 AI 自动建模、在测试阶段引入智能用例生成，我们将开发人员从“体力密集型”的代码搬运中解放出来，使其能够更专注于 Redis 缓存策略设计与高并发架构调优。

同时，本方案清醒地认识到 AI 带来的代码冗余与潜在安全漏洞风险。通过设立“AI 代码与安全审查员”角色以及强制的“Human-in-the-loop”复核流转节点，我们在追求极致开发效率的同时，死守了软件质量与数据安全的底线。在未来的项目演进中，我们期望进一步探索 AI Agent 在系统运行维护阶段（如日志自动寻根与流量自适应限流）的深度融合。

七、 参考文献

- [1] Harding, C., & Pomeroy, Peter. (2024). *Coding on Copilot: 2023 Data Suggests Downward Pressure on Code Quality*. GitClear Research. (行业真实数据报告，深度剖析了 AI 辅助编程导致代码重复率与代码流动率上升的风险)
- [2] Schäfer, M., Nadi, S., et al. (2024). *An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation*. IEEE Transactions on Software Engineering. (软件工程顶刊论文，验证了利用 LLM 自动生成单元测试与边界值的有效性与效率提升)
- [3] Pearce, H., Ahmad, B., et al. (2022). *Asleep at the Keyboard? Assessing the Security*

of GitHub Copilot's Code Contributions. IEEE Symposium on Security and Privacy (S&P). (安全领域顶级会议论文, 揭示了 AI 代码生成模型在特定场景下产生安全漏洞的风险)

[4] 中国信息通信研究院. (2024). 《*智能化软件工程技术和应用要求 第3部分: 智能测试能力*》. (国内权威行业标准, 为项目规范 AI 测试覆盖率与测试分析能力提供了准则)